

A BEGINNER'S FIELD GUIDE

Building a VR Classroom

How to rebuild the **VRClass** project in Unity — a walk-in virtual study room for the Meta Quest, with a working drawing board, grabbable objects and a safety warning system.

CREATED BY Sara Mjeshtri & Jorida Lilollari

Unity 2022.3 LTS

Meta XR SDK

Meta Quest 2 / 3 / Pro

C# × 6 scripts

~12-20 hours

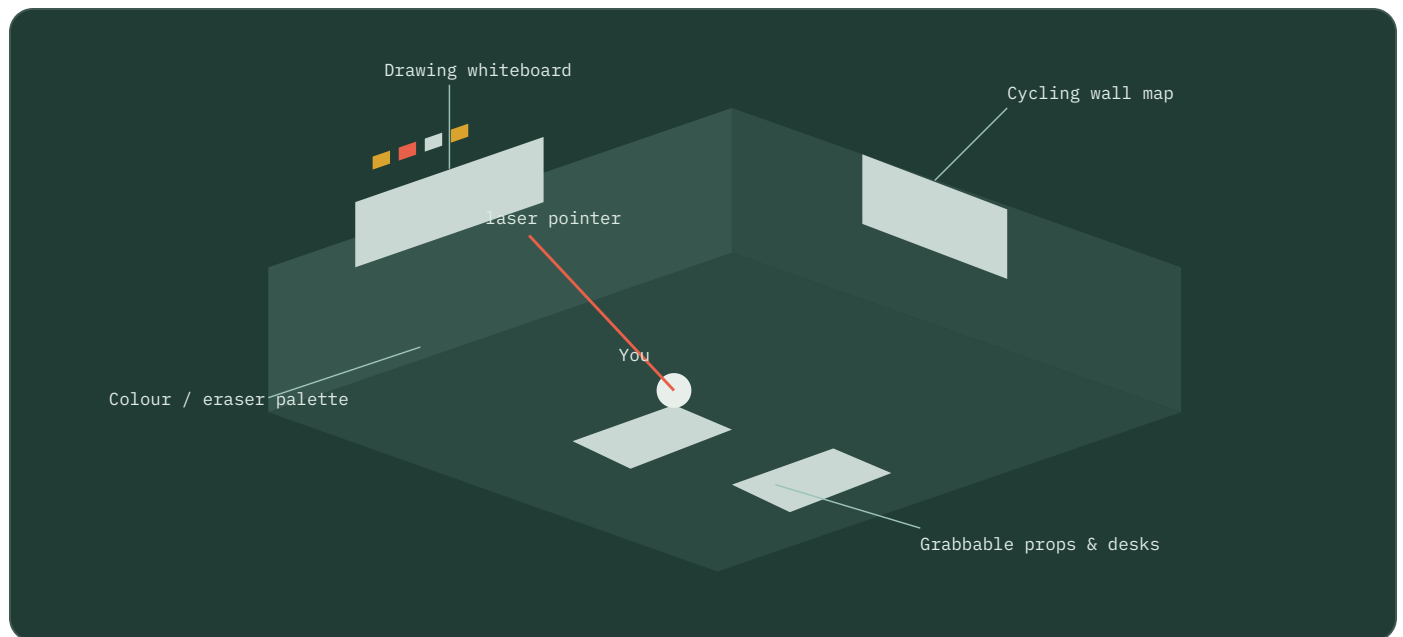


FIG. 0 The finished room. Everything you point at with the laser reacts — you draw on the board, pick a colour, grab a chair, or flip the map.

What this guide is

A step-by-step rebuild of a real student project, written for someone who has never opened Unity.

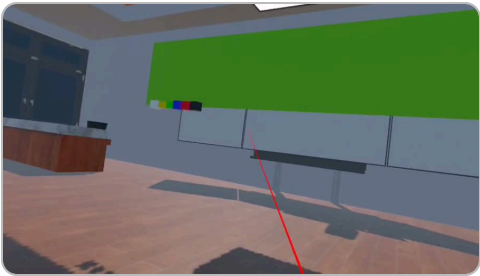
This book explains how the **VRClass** project was built: a virtual classroom you can walk around in with a Meta Quest headset. Inside the room you can draw on a whiteboard with different colours, pick things up from across the room, flip through images on a wall map, and get a warning buzz when you are about to walk into a desk.

You do *not* need to know Unity or C# already. Every part starts from menus and buttons. When a script is involved, the important pieces are printed in full and explained line by line. By the end you will have the same project running on a headset.

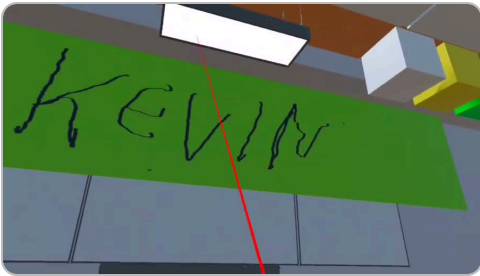
Here is the finished project, running on a Quest headset:



The room — walk around it in VR.



The board and the colour / eraser palette.



Drawing on the board with the laser.

The short version of the idea: build a room out of 3D models, drop in a camera rig that represents the headset and hands, then write a handful of small C# scripts that shoot an invisible ray out of the controller and make things happen wherever it lands.

What you need

THING	WHICH ONE	NOTES
A computer	Windows 10/11 or macOS	8 GB RAM minimum, 16 GB comfortable. ~40 GB free disk.
A VR headset	Meta Quest 2, Quest 3, Quest Pro or Quest 3S	This project targets stand-alone Quest (Android). A link cable helps but is not required.
A USB-C cable	The charging cable is fine	Used once to install the app onto the headset.
A Meta account	Free	Needed to put the headset into developer mode.
3D models	Free from Sketchfab / Poly Pizza	Full list in Appendix C. All are free, "download-able" models.

The exact versions this project uses

Project facts		
Unity Editor		2022.3.62f3 (LTS)
Render pipeline		Built-in
Colour space		Linear
VR SDK	com.meta.xr.sdk.all	201.0.0
Input	OVRInput (from the Meta SDK)	
Build target	Android (stand-alone Quest)	
Scene	Assets/Scenes/SampleScene.unity	
Custom scripts	6 gameplay + 1 editor tool	

You can use a newer Unity 6 LTS if you prefer — the steps are almost identical — but if you want to match this project exactly, install **2022.3.62f3**.

CONTENTS

The build, in order

01	Install the tools	Unity Hub, Unity, headset setup
02	Create the project & learn the Editor	Panels, GameObjects, the Play button
03	Turn it into a VR project	Meta XR SDK, the Camera Rig, build settings
04	Build the room	Import models, fix materials, light it
05	The laser pointer	ControllerRayFollower — the hub of everything
06	Grabbing objects	VRGrabbableObject, Rigidbody, colliders
07	The drawing whiteboard	WhiteboardDrawer — painting on a texture
08	The colour palette	ColorPickerPanel + ColorPanelSetup
09	The cycling wall map	MapImageCycler

PART 1

Install the tools

Unity Hub manages versions and projects. Unity is the editor itself. The headset needs a one-time setup.

1.1 Unity Hub

Go to unity.com/download and install **Unity Hub**. The Hub is a small launcher — it is not the editor. It keeps a list of your projects and the editor versions installed on your machine, and it lets you have several versions side by side.

When you first open the Hub it asks you to sign in with a free **Unity ID** and pick a licence. Choose the **Personal** licence — it is free for students and hobby projects.

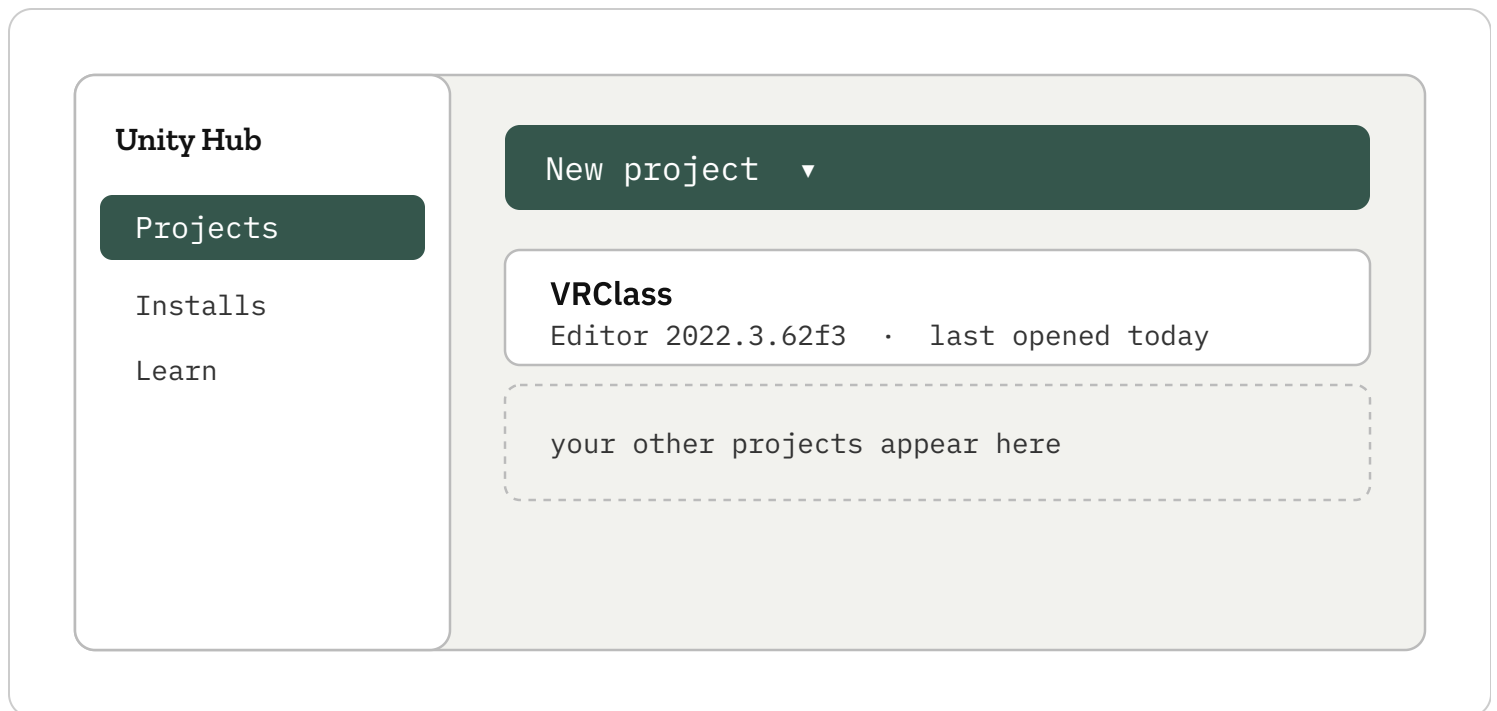


FIG. 1.1 The Hub. **Projects** lists your work; **Installs** is the tab where editor versions live; **Learn** has sample projects.

1.2 Install the Unity Editor

- 1 In the Hub, open **Installs** → **Install Editor**.
- 2 Pick **2022.3.62f3** (under the "Official releases" / LTS list). If that exact patch is gone, any `2022.3.x` works.
- 3 On the "Add modules" screen, tick **Android Build Support**. Expand it and also tick **OpenJDK** and **Android SDK & NDK Tools**. This is what lets Unity build an app the Quest can run.
- 4 If you also want to test on a PC screen, leave **Windows / Mac Build Support** ticked too.
- 5 Click **Install** and wait. It is a big download (several GB).

DON'T SKIP

Android Build Support is the one module people forget. Without it, Unity cannot make a `.apk` and the headset menus in Part 3 stay greyed out. You can add it later from **Installs** → (gear icon) → **Add modules**, but it is easier now.

Add modules

- ☒ Android Build Support
 - ☒ OpenJDK
 - ☒ Android SDK & NDK Tools
- ☐ Windows Build Support (IL2CPP)
- ☐ Documentation

Install

FIG. 1.2 Tick the three Android items. Everything else is optional.

1.3 Connecting the headset to your computer

Unity talks to the Quest through a tool called **ADB** (Android Debug Bridge). It is bundled with the Android module you installed in §1.2, so you rarely install it by hand — but getting the computer to see the headset works a little differently on each operating system.

Three things are the same everywhere:

- **Developer Mode** turned on for the headset — a one-time toggle in your Meta account's device settings (also available from the Meta Quest Developer Hub desktop app). You may be asked to create a free "developer organisation" first; any name works.
- A **data-capable** USB-C cable — many cheap cables only carry power.
- Accepting the **Allow USB debugging** prompt that appears inside the headset the first time you plug in.

YOUR COMPUTER	WHAT TO SET UP	CHECK IT WORKS
Windows	Install the Meta Quest ADB Drivers — a small <code>.msi</code> from the Meta developer downloads page — so Windows recognises the headset. Optional but handy: Meta Quest Developer Hub (MQDH) , a desktop app that shows the connection, logs and screen mirroring.	Open PowerShell and run <code>adb devices</code> . Your headset's serial number should appear with <code>device</code> next to it.
macOS	No driver needed. If <code>adb</code> is not found in Terminal, install it with <code>brew install android-platform-tools</code> , or use the copy Unity ships at <code>.../PlaybackEngines/AndroidPlayer/SDK/platform-tools/</code> . MQDH also has a Mac version.	In Terminal : <code>adb devices</code> .
Linux	Install platform-tools (<code>sudo apt install android-tools-adb</code> on Debian/Ubuntu). Add a udev rule so a normal user can see the headset: create <code>/etc/udev/rules.d/51-android.rules</code> containing <code>SUBSYSTEM=="usb", ATTR{idVendor}=="2833", MODE="0666", GROUP="plugdev"</code> then <code>sudo udevadm control --reload-rules && sudo udevadm trigger</code> . Add yourself to the <code>plugdev</code> group and log out/in.	In Terminal : <code>adb devices</code> .

Meta's USB vendor ID is **2833** — useful for the Linux udev rule and for spotting the device in `lsusb` / Device Manager.

- 1 Plug the headset into the computer with the USB-C cable.
- 2 Put the headset on and approve **Allow USB debugging** (tick "Always allow from this computer").
- 3 On the computer, run `adb devices`. The first time it may say `unauthorized` — that means step 2's prompt has not been accepted yet.
- 4 Once it shows `device`, Unity's **Build & Run** (Part 11) will find the headset automatically.

GOING WIRELESS (NO CABLE)

Connect once by cable, then run `adb tcpip 5555`. Unplug the cable and run `adb connect <headset-IP>` (find the IP in the headset under **Settings** → **Wi-Fi** → **(your network)**). Build & Run now works over Wi-Fi. You need to re-connect after the headset reboots.

IF THE HEADSET NEVER SHOWS UP

Try a different cable and port (it must carry data, not just power). Run `adb kill-server` then `adb start-server`. On Windows, make sure only one ADB is running — Unity's and MQDH's copies can fight over the connection. Quest Link / Air Link being active can also block it; quit the Link app first.

PART 2

Create the project & learn the Editor

Five panels do almost everything. Learn what each one is for and the rest of the book is just "put thing here, set value there."

2.1 Make the project

- 1 Hub → **New project**.
- 2 Editor version: **2022.3.62f3**.
- 3 Template: **3D (Built-In Render Pipeline)**. *Not* URP — this project uses Built-in.
- 4 Name it **VRClass**, choose a folder, **Create project**.

Why Built-in? It is the simplest pipeline and the Meta samples all work with it out of the box. The trade-off is fewer fancy graphics features, which a classroom scene does not need.

2.2 The Editor window

When the project opens you get one big window split into panels. Here is what each area is and how this book refers to it.

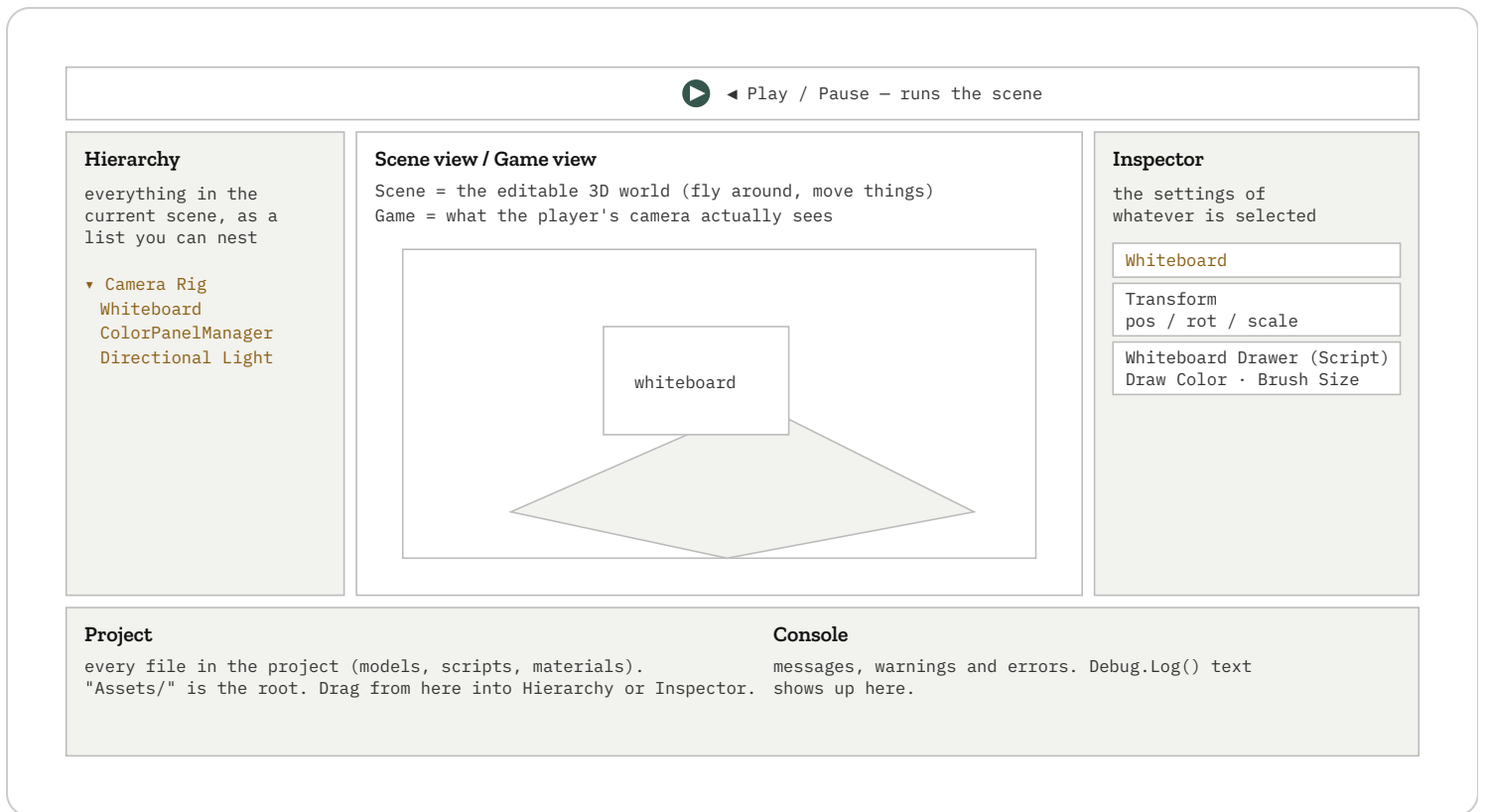


FIG. 2.1 The five panels. The whole workflow is: select something in the **Hierarchy**, change it in the **Inspector**, drag files in from the **Project**, press **Play**, read the **Console**.

WORD	MEANS
Scene	One "level" or screen. This project has one: <code>SampleScene</code> .
GameObject	Anything that exists in the scene — a light, a model, an empty marker. It is basically an empty container.
Component	A behaviour you attach to a GameObject. A <i>Mesh Renderer</i> makes it visible; a <i>Rigidbody</i> gives it physics; <i>your script</i> is also a component.
Transform	Every GameObject's position, rotation and scale. Always the first thing in the Inspector.
Prefab	A saved GameObject you can reuse. The Meta Camera Rig is a prefab.
Material	How a surface looks — its colour, texture and shininess.
Asset	Any file in the <code>Assets</code> folder.

THE MENTAL MODEL

A GameObject is a peg-board. Components are the tools you clip onto it. A script is a component you wrote. "Attach the script to the Whiteboard" just means: select the Whiteboard, click **Add Component**, choose your script.

Turn it into a VR project

Install the Meta XR SDK, drop in the Camera Rig that stands in for the headset and hands, and point the build settings at Android.

3.1 Install the Meta XR SDK

The Meta XR SDK (also called the "Oculus Integration" / "Meta XR All-in-One SDK") gives you the headset tracking, the controller input system (`OVRInput`), hand tracking, and a library of ready-made "Building Blocks."

- 1 Get it from the **Unity Asset Store** page for "*Meta XR All-in-One SDK*" (free) — click **Add to My Assets**.
- 2 Back in Unity: **Window → Package Manager**. Switch the top-left dropdown to **My Assets**, find the SDK, click **Download** then **Import**.
- 3 Unity will pop up several prompts asking to change project settings for XR ("restart", "enable the new input system", "use the recommended settings"). Accept them all. It will restart the editor once or twice.
- 4 Open **Meta → Tools → Project Setup Tool** and click **Fix All** then **Apply All**. This flips a long list of Android and XR settings to what the Quest needs.

In this project the SDK shows up in `Packages/manifest.json` as **com.meta.xr.sdk.all** version **201.0.0**. If you install a newer version the menus may move slightly but the ideas are the same.

WHAT "XR PLUG-IN MANAGEMENT" IS

Under **Edit → Project Settings → XR Plug-in Management** there are two tabs: a desktop one and an **Android** one (the little robot). On the Android tab, tick **Oculus**. That is the piece that actually talks to the Quest hardware. The Project Setup Tool usually does this for you — just confirm the tick is there.

XR Plug-in Management



Android tab is selected

Plug-in Providers:



Oculus



OpenXR



Unity Mock HMD

FIG. 3.1 On the Android tab, **Oculus** is ticked. This matches the project's XR/Loaders setup.

3.2 Add the Camera Rig

Right now the scene has a plain **Main Camera** that does not know about a headset. Replace it with Meta's rig.

- 1 Delete the **Main Camera** in the Hierarchy.
- 2 Open **Meta → Tools → Building Blocks**. A panel of tiles appears.
- 3 Drag the **Camera Rig** tile into the Scene view (or click it). This adds a GameObject the project calls `[BuildingBlock] Camera Rig`.
- 4 Set its Transform position to where the player should stand — roughly the middle of the room, `y = 0` (floor level).

The rig is a small tree of GameObjects. You will refer to these names constantly in the scripts:

■ Hierarchy – the Camera Rig

```
▼ [BuildingBlock] Camera Rig           # has OVRManager + OVRCameraRig
  ▼ TrackingSpace
    CenterEyeAnchor                     # the player's head / camera
    LeftEyeAnchor   RightEyeAnchor
    ▼ LeftHandAnchor                    # follows the left controller
      LeftControllerAnchor
    ▼ RightHandAnchor                   # follows the right controller
      RightControllerAnchor             # ← the laser lives here
```

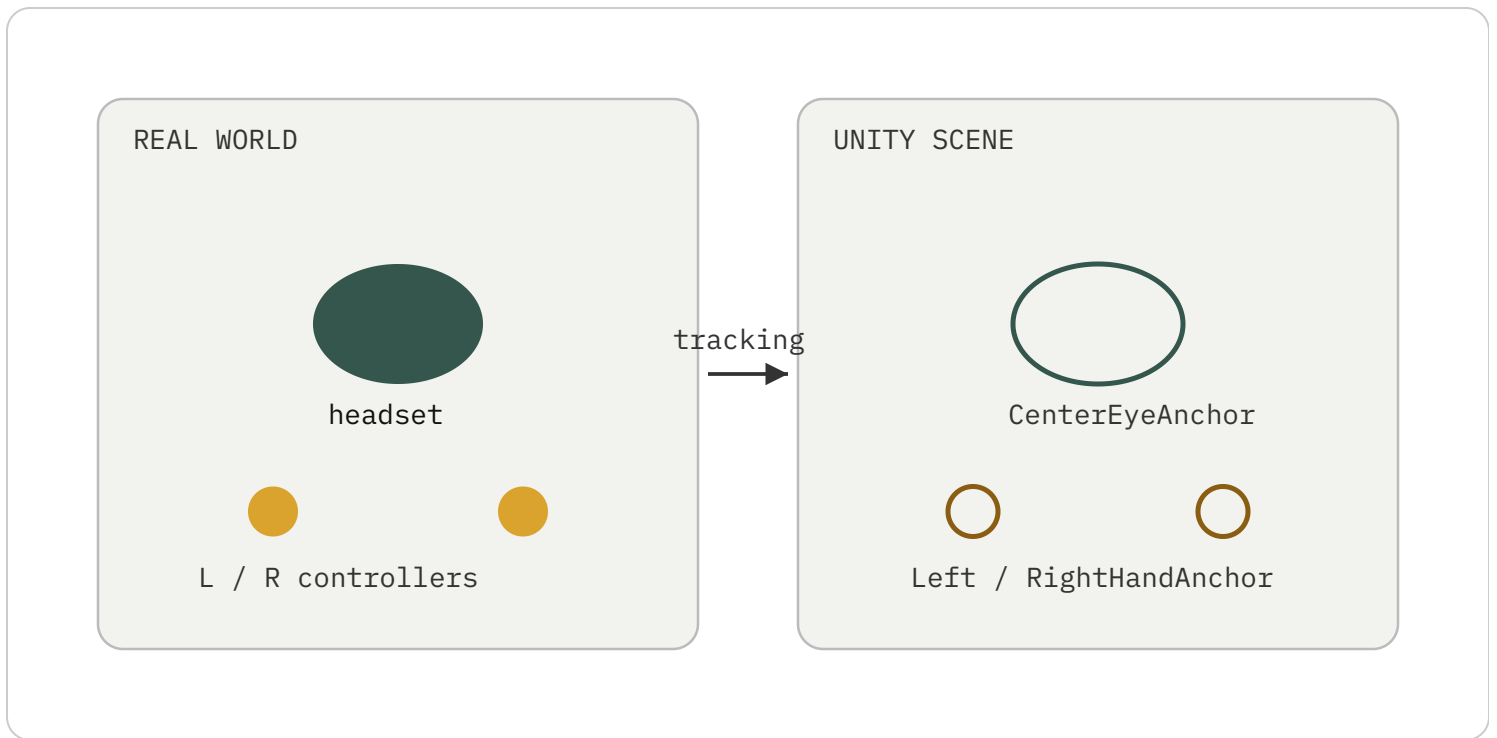


FIG. 3.2 The rig's anchors are invisible GameObjects that Unity keeps glued to your real head and hands every frame. Your scripts read their position and rotation.

3.3 Build settings & project config

- 1 **File** → **Build Settings** → select **Android**, click **Switch Platform** (this takes a while the first time).
- 2 Click **Add Open Scenes** so `SampleScene` is in the build list.
- 3 **Edit** → **Project Settings** → **Player** → **Other Settings**: set **Minimum API Level** to Android 10 (API 29) or higher, **Scripting Backend** to **IL2CPP**, and tick **ARM64** (untick ARMv7). The Quest requires ARM64.
- 4 Open **Meta** → **Tools** → **Project Setup Tool** once more and clear any remaining warnings.

This project also has hand-tracking support switched on in `Assets/Oculus/OculusProjectConfig.asset`. It is not required for the tutorial features, but leaving it on does no harm.

CHECKPOINT

Press **Play** with the headset connected (Quest Link) or with the Meta XR Simulator. You should be standing in an empty scene able to look around. If yes, VR is working — time to build the room.

Build the room

Import a classroom model, hang its textures back on, add the props, and light it.

4.1 Get the models

Every object in VRClass is a free downloadable 3D model in **FBX** format. The room shell is a model called "*Brand new Study Room*"; the rest are props (a bookcase, books, an inkwell, a bin, a car, a Quest headset model, a bouquet). Full credits are in Appendix C.

When you download a model from Sketchfab you get a `.zip` containing a `source/` folder (the `.fbx`) and a `textures/` folder (`.png` images). Keep both.

Import

- 1 In the **Project** panel make a folder, e.g. `Assets/Classroom`.
- 2 Drag the `.fbx` and all the `.png` texture files into it.
- 3 Select the `.fbx`. In the Inspector, on the **Materials** tab, set **Material Creation Mode** to *Standard* and click **Extract Materials...** — choose the same folder. Now each surface has its own editable material asset.

THE CLASSIC PROBLEM

Fresh-imported models often look flat grey or plain white. FBX files store *which* texture goes where by name, but Unity does not always reconnect them. You have to open each material and drag its texture into the **Base Map** slot. In this project there are ~20 materials, so...

4.2 An editor tool to reconnect textures fast

The project includes a small **Editor script** that loops over every material in `Assets/Classroom` and plugs in the matching `_BaseColor` texture by name, plus makes anything called "Glass" transparent. An editor script is code that adds a menu item to Unity itself — it never runs in the game. It must live in a folder called `Assets/Editor`.

ASSETS/EDITOR/FIXMATERIALS.CS — ADDS THE MENU "TOOLS > FIX EVERYTHING"

```
using UnityEngine;
using UnityEditor;
using System.Collections.Generic;

public class FixMaterials : MonoBehaviour
{
    [MenuItem("Tools/Fix Everything")]
    static void FixEverything()
```

```

{
    // material name -> texture file name (without extension)
    Dictionary<string, string> matToTex = new Dictionary<string, string>()
    {
        { "Chair",      "Chair_Chair_BaseColor" },
        { "Door_",      "Door_Door_BaseColor" },
        { "Floor",       "Floor_DefaultMaterial_BaseColor" },
        { "Walls",       "Walls_Walls_Diffuse" },
        { "Map_Image",   "Map_on_hte_Wall_Map_Image_BaseColor" },
        // ...one line per surface in the model...
    };

    // find every material asset under Assets/Classroom
    string[] matGuids = AssetDatabase.FindAssets("t:Material", new[] { "Assets/Classroom" });

    foreach (string guid in matGuids)
    {
        string matPath = AssetDatabase.GUIDToAssetPath(guid);
        Material mat = AssetDatabase.LoadAssetAtPath<Material>(matPath);
        if (mat == null || !matToTex.ContainsKey(mat.name)) continue;

        // find the texture whose name we mapped, and plug it into Base Map
        string texName = matToTex[mat.name];
        string[] texGuids = AssetDatabase.FindAssets(texName + " t:Texture2D");
        if (texGuids.Length > 0)
        {
            Texture2D tex = AssetDatabase.LoadAssetAtPath<Texture2D>(
                AssetDatabase.GUIDToAssetPath(texGuids[0]));
            mat.SetTexture("_BaseMap", tex);
            mat.SetColor("_BaseColor", Color.white);
        }

        // anything called "Glass" becomes see-through
        if (mat.name.Contains("Glass"))
        {
            mat.SetFloat("_Surface", 1); // transparent mode
            mat.SetColor("_BaseColor", new Color(0.8f, 0.9f, 1f, 0.25f));
            mat.renderQueue = 3000;
        }
        EditorUtility.SetDirty(mat);
    }
    AssetDatabase.SaveAssets();
    Debug.Log("Materials fixed.");
}
}

```

- `[MenuItem("Tools/Fix Everything")]` is what puts **Tools → Fix Everything** in the top menu bar.
- `AssetDatabase.FindAssets` is Unity's way of searching your project files from code.
- You run it once after importing, check the room, then you never need it again.

Adapting it: the dictionary keys must match *your* material names and the values must match *your* texture file names. Select a material, read its name at the top of the Inspector; select a texture, read its name in the Project panel. If you use Unity's Built-in pipeline instead of URP, replace `"_BaseMap"/"_BaseColor"` with `"_MainTex"/"_Color"`.

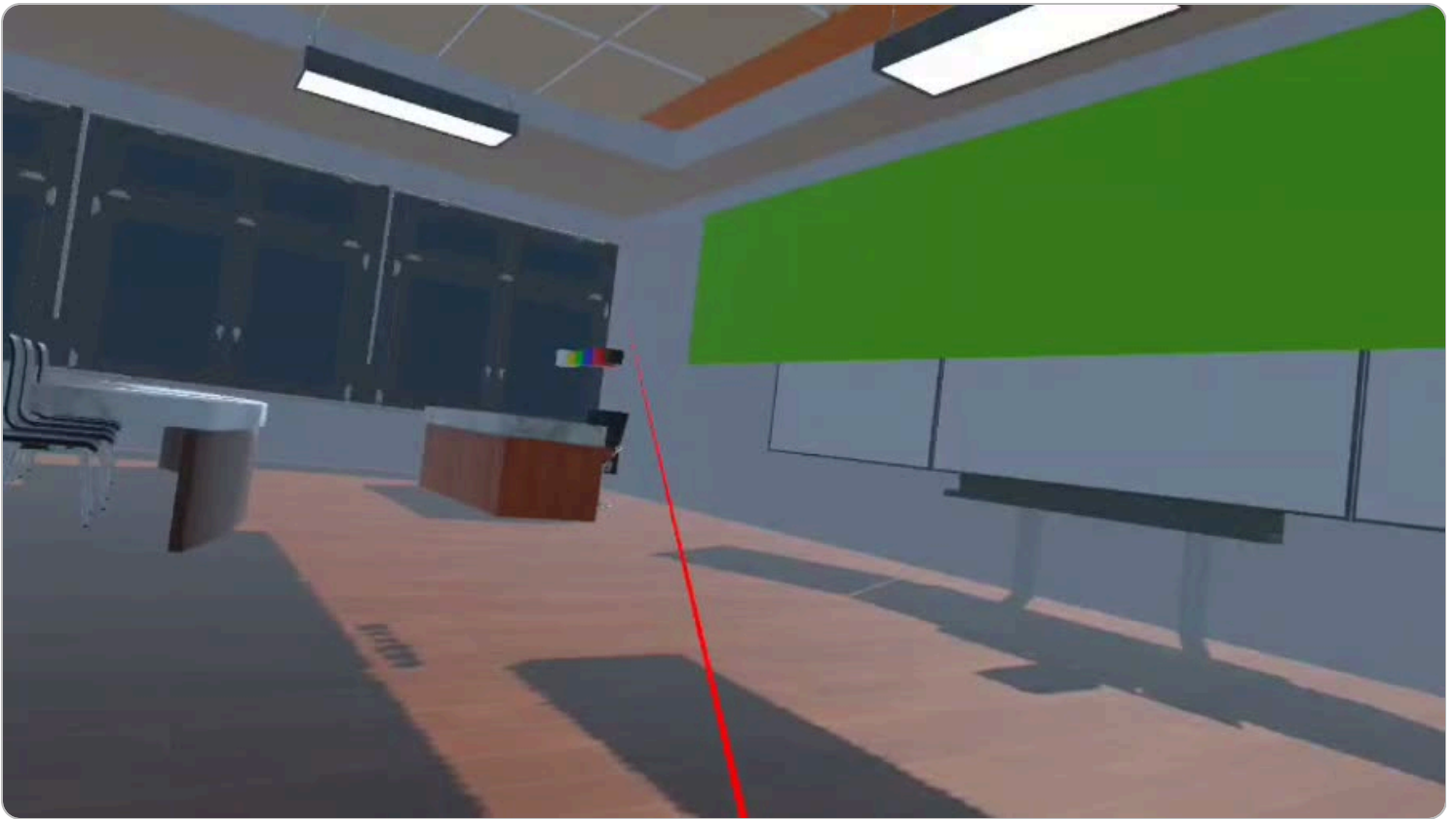


FIG. 4.1 The room once the models are imported, their textures reconnected (Tools > Fix Everything) and the lighting baked. This is the build shown in the video this guide was written from.

4.3 Assemble the scene

- 1 Drag the classroom FBX from the Project panel into the Hierarchy. Set its Transform to `position (0,0,0)`.
- 2 Walk the camera through it in the Scene view. Scale the room if it feels too small/large — a real classroom is about 6×8 metres and the player is ~1.7 m tall.
- 3 Drag in the prop models (bookcase, books, bin...) and place them on desks and shelves.
- 4 Add a **plane** or **quad** for the whiteboard on the front wall (GameObject → 3D Object → Quad). Name it **Whiteboard**. This is the surface you will draw on in Part 7.

■ Hierarchy – target layout for the finished scene

```
▼ [BuildingBlock] Camera Rig
  Directional Light
▼ Brand new Study Room      # the imported room model
  Walls · Floor · Windows · Door · Desks · Chairs · Map_Image ...
Whiteboard                  # quad + MeshCollider + WhiteboardDrawer
▼ ColorPanelManager         # empty, holds ColorPanelSetup
  ColorPanel_Black · _Red · _Blue · _Green · _Yellow · _Eraser
▼ Props
  BookCase · BookA · Inkwell · Big bin · Bouquet · Quest2 ... # each: Collider + Rigidbody +
VRGrabbableObject
▼ Canvas (World Space)
  WarningText               # the "step back" warning
EventSystem
```

4.4 Lighting

- Keep the default **Directional Light** as sunlight through the windows. Rotate it until shadows look right.
- **Window → Rendering → Lighting**: click **Generate Lighting** to bake soft shadows and bounce light. On Quest, baked lighting is much cheaper than real-time.
- Set the room's static objects (walls, floor, desks) to **Static** (tick the box top-right of the Inspector) so they get baked.

Colour space: this project uses **Linear** (Edit → Project Settings → Player → Other Settings → Color Space). It makes lighting look more realistic. Set it early — changing it later re-imports every texture.

PART 5

The laser pointer

One script on one object. It draws the laser, shoots a ray every frame, and tells whatever it hits what to do. Every other feature plugs into this.

This project does **not** use Unity's XR Interaction Toolkit or Meta's interaction prefabs for pointing. It does the simplest possible thing: a script called **ControllerRayFollower** that

1. moves itself to the right controller's position every frame,
2. casts a ray straight forward,
3. looks at what the ray hits and calls a method on it,
4. draws a coloured line so you can see where you are pointing.

5.1 Set it up

- 1 Create an empty GameObject, name it **LaserPointer**. Make it a child of `RightHandAnchor` (or leave it loose — the script repositions it anyway).
- 2 Add Component → **Line Renderer**. The script configures its width and colour, so you can leave the defaults.
- 3 Add Component → **Controller Ray Follower** (your script).

5.2 How the ray dispatch works

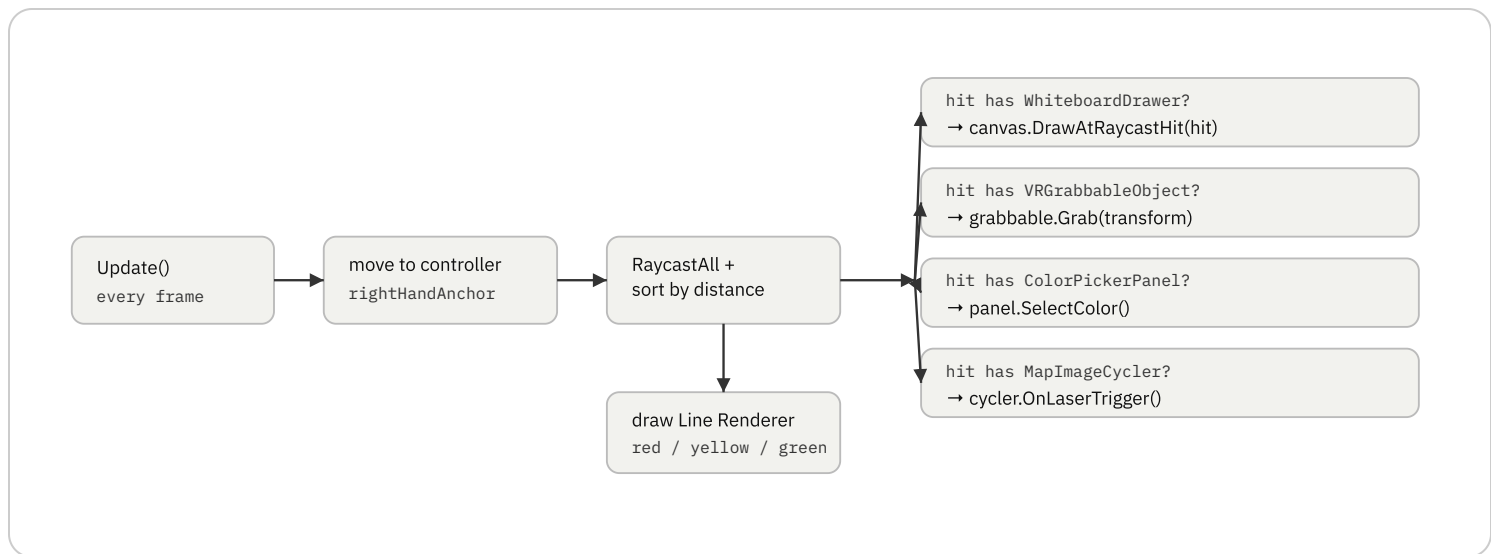


FIG. 5.1 The dispatcher. `Physics.RaycastAll` returns everything under the ray; the script sorts them nearest-first and acts on the first one it recognises, then breaks.

5.3 The script

Read the comments — they explain each block. The three input lines near the top are the whole controller-input story: `OVRInput.GetDown` is "trigger pressed this frame", `GetUp` is "released this frame", `Get` is "held right now".

ASSETS/CONTROLLERRAYFOLLOWER.CS — ATTACH TO THE LASERPOINTER OBJECT

```
using UnityEngine;

public class ControllerRayFollower : MonoBehaviour
{
    private LineRenderer lineRenderer;
    private OVRCameraRig rig;

    private VRGrabbableObject grabbedObject;    // what we're currently holding
    private VRGrabbableObject hoveredGrabbable; // what the laser is on right now
    private ColorPickerPanel lastHovered;

    [Header("Ray Settings")]
```

```

[Tooltip("How far the laser reaches (bigger = grab things further away)")]
public float rayDistance = 50f;

void Start()
{
    rig = FindObjectOfType<OVRCameraRig>();

    lineRenderer = GetComponent<LineRenderer>();
    if (lineRenderer == null)
        lineRenderer = gameObject.AddComponent<LineRenderer>();

    lineRenderer.startWidth = lineRenderer.endWidth = 0.005f;    // 5 mm thick
    lineRenderer.positionCount = 2;
    lineRenderer.material = new Material(Shader.Find("Sprites/Default"));
    lineRenderer.startColor = lineRenderer.endColor = Color.red;
}

void Update()
{
    if (rig == null) { rig = FindObjectOfType<OVRCameraRig>(); return; }

    // 1. glue this object to the right controller
    Transform anchor = rig.rightHandAnchor;
    transform.position = anchor.position;
    transform.rotation = anchor.rotation;

    // 2. build the ray and read the trigger
    Ray ray = new Ray(transform.position, transform.forward);
    Vector3 endPoint = transform.position + transform.forward * rayDistance;

    bool triggerDown = OVRInput.GetDown(OVRInput.Button.PrimaryIndexTrigger,
OVRInput.Controller.RTouch);
    bool triggerUp    = OVRInput.GetUp   (OVRInput.Button.PrimaryIndexTrigger,
OVRInput.Controller.RTouch);
    bool triggerHeld  = OVRInput.Get     (OVRInput.Button.PrimaryIndexTrigger,
OVRInput.Controller.RTouch);

    // 3. release a held object the moment the trigger comes up
    if (triggerUp && grabbedObject != null)
    {
        grabbedObject.Release();
        grabbedObject = null;
    }

    // 4. while holding something, don't raycast at all – just draw a green line
    if (grabbedObject != null)
    {
        lineRenderer.startColor = lineRenderer.endColor = Color.green;
        lineRenderer.SetPosition(0, transform.position);
        lineRenderer.SetPosition(1, transform.position + transform.forward * rayDistance);
        return;
    }

    // 5. cast the ray, nearest hit first
    RaycastHit[] hits = Physics.RaycastAll(ray, rayDistance);
    System.Array.Sort(hits, (a, b) => a.distance.CompareTo(b.distance));

    VRGrabbableObject grabbableThisFrame = null;

```

```

foreach (RaycastHit hit in hits)
{
    GameObject hitObj = hit.collider.gameObject;

    // --- a whiteboard? paint while the trigger is held ---
    WhiteboardDrawer canvas = hitObj.GetComponent<WhiteboardDrawer>();
    if (canvas != null)
    {
        endPoint = hit.point;
        if (triggerHeld) canvas.DrawAtRaycastHit(hit);
        else            canvas.StopDrawing();
        break;
    }

    // --- a grabbable? remember it; grab on trigger press ---
    VRGrabbableObject grabbable = hitObj.GetComponent<VRGrabbableObject>();
    if (grabbable != null)
    {
        grabbableThisFrame = grabbable;
        endPoint = hit.point;
        if (triggerDown) { grabbedObject = grabbable; grabbable.Grab(transform); }
        break;
    }

    // --- a colour panel? select on trigger press ---
    ColorPickerPanel picker = hitObj.GetComponent<ColorPickerPanel>();
    if (picker != null && triggerDown) { picker.SelectColor(); endPoint = hit.point; break; }

    // --- the wall map? advance on trigger press ---
    MapImageCycler cycler = hitObj.GetComponent<MapImageCycler>();
    if (cycler != null && triggerDown) { cycler.OnLaserTrigger(); endPoint = hit.point; break; }
}

}

// 6. hover highlight for grabbables (enter / exit)
if (grabbableThisFrame != hoveredGrabbable)
{
    if (hoveredGrabbable != null) hoveredGrabbable.OnHoverExit();
    if (grabbableThisFrame != null) grabbableThisFrame.OnHoverEnter();
    hoveredGrabbable = grabbableThisFrame;
}

// 7. laser colour feedback: red = nothing yellow = on a grabbable
lineRenderer.startColor = lineRenderer.endColor =
    (grabbableThisFrame != null) ? Color.yellow : Color.red;

lineRenderer.SetPosition(0, transform.position);
lineRenderer.SetPosition(1, endPoint);
}
}

```

WHY THIS DESIGN

Putting the "what does the ray do" logic in one place means each feature script (whiteboard, grabbable, map...) only has to expose one public method — `SelectColor()` , `Grab()` , `OnLaserTrigger()` . You can add a new interactive object later just by giving it a script with one method and adding an `if` here.

Left hand too? Duplicate the LaserPointer, parent it to LeftHandAnchor, and change `rightHandAnchor` → `leftHandAnchor` and `RTouch` → `LTouch` in a copy of the script (or add a public bool to switch).

PART 6

Grabbing objects

Add one script to anything you want to pick up. It holds the object out in front of the controller at whatever distance you grabbed it — near or across the room.

6.1 What a grabbable needs

Select a prop and add three components:

COMPONENT	SETTINGS	WHY
Collider (Box is fine)	<code>Is Trigger</code> = off	The ray can only hit solid colliders. A trigger collider is invisible to raycasts.
Rigidbody	default	Lets the script move it with physics and lets it fall when released.
VRGrabbableObject (script)	<code>Follow Speed</code> ≈ 20 <code>Highlight Material</code> = a bright material (optional)	The grab behaviour itself.

The script enforces the first two for you with `[RequireComponent]` — Unity adds a Collider and Rigidbody automatically if they are missing.

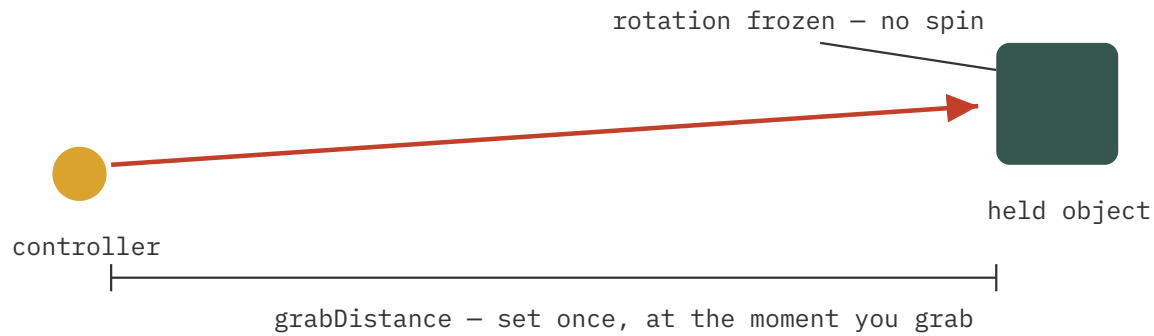


FIG. 6.1 "Far grab." At the moment you press the trigger the script records how far away the object is, then every physics step it pushes the object toward $\text{controller.position} + \text{controller.forward} \times \text{grabDistance}$.

ASSETS/VRGRABBABLEOBJECT.CS – ATTACH TO EACH PICKUP-ABLE PROP

```
using UnityEngine;

[RequireComponent(typeof(Collider))]
[RequireComponent(typeof(Rigidbody))]
public class VRGrabbableObject : MonoBehaviour
{
    [Header("Grab Settings")]
    public float followSpeed = 20f;           // how hard it chases the target point

    [Header("Highlight Settings")]
    public Material highlightMaterial;        // shown while the laser hovers

    private Rigidbody rb;
    private Renderer rend;
    private Material originalMaterial;

    private bool isGrabbed;
    private Transform grabAnchor;            // the controller
    private float grabDistance;              // distance recorded at grab time
    private Quaternion grabRotationOffset;   // world rotation, frozen

    void Awake()
    {
        rb = GetComponent<Rigidbody>();
        rend = GetComponentInChildren<Renderer>();
        if (rend != null) originalMaterial = rend.material;
        GetComponent<Collider>().isTrigger = false; // must be solid for the ray
    }

    void FixedUpdate()                        // physics runs here, not Update
    {
        if (!isGrabbed) return;

        // target = a point straight out in front of the controller
    }
}
```

```

    Vector3 target = grabAnchor.position + grabAnchor.forward * grabDistance;

    // move toward it by setting velocity (smooth, physics-friendly)
    rb.velocity      = (target - transform.position) * followSpeed;
    rb.angularVelocity = Vector3.zero;           // no spinning
    transform.rotation = grabRotationOffset;
}

// ---- called by ControllerRayFollower ----

public void OnHoverEnter()
{
    if (highlightMaterial != null && rend != null) rend.material = highlightMaterial;
}
public void OnHoverExit()
{
    if (rend != null) rend.material = originalMaterial;
}

public void Grab(Transform anchor)
{
    if (isGrabbed) return;
    isGrabbed = true;
    grabAnchor = anchor;

    grabDistance = Mathf.Max(0.1f, Vector3.Distance(anchor.position, transform.position));
    grabRotationOffset = transform.rotation;

    rb.useGravity      = false;           // don't fall while held
    rb.isKinematic     = false;
    rb.freezeRotation = true;
    OnHoverExit();           // drop the highlight
}

public void Release()
{
    if (!isGrabbed) return;
    isGrabbed = false;
    grabAnchor = null;
    rb.useGravity      = true;           // fall naturally
    rb.freezeRotation = false;
}
}

```

- **Why FixedUpdate ?** Anything that touches a Rigidbody belongs in `FixedUpdate` , which runs on the physics clock. Using `Update` makes grabbed objects jitter.
- **Why set velocity instead of position ?** Teleporting a Rigidbody with `position` makes it pass through walls and floors. Setting velocity lets the physics engine resolve collisions.
- **The `Mathf.Max(0.1f, ...)`** keeps the object at least 10 cm away so it never ends up inside your virtual hand.

TEST

Press Play, point at a prop (laser turns **yellow**), pull the trigger (laser turns **green**), wave the controller, release. The prop should drop and settle.

PART 7

The drawing whiteboard

The board is one flat quad. Its surface is a picture you edit pixel by pixel while the laser drags across it.

7.1 The idea

The whiteboard's material shows a **Texture2D** — a 2048×2048 image the script creates in memory and fills with white. When the laser hits the board, the hit info includes `hit.textureCoord`: the exact **UV** coordinate (0–1 across, 0–1 up) where the ray landed. Multiply that by 2048 and you have the pixel to colour. Paint a little filled circle there, call `texture.Apply()`, and the drawing appears.

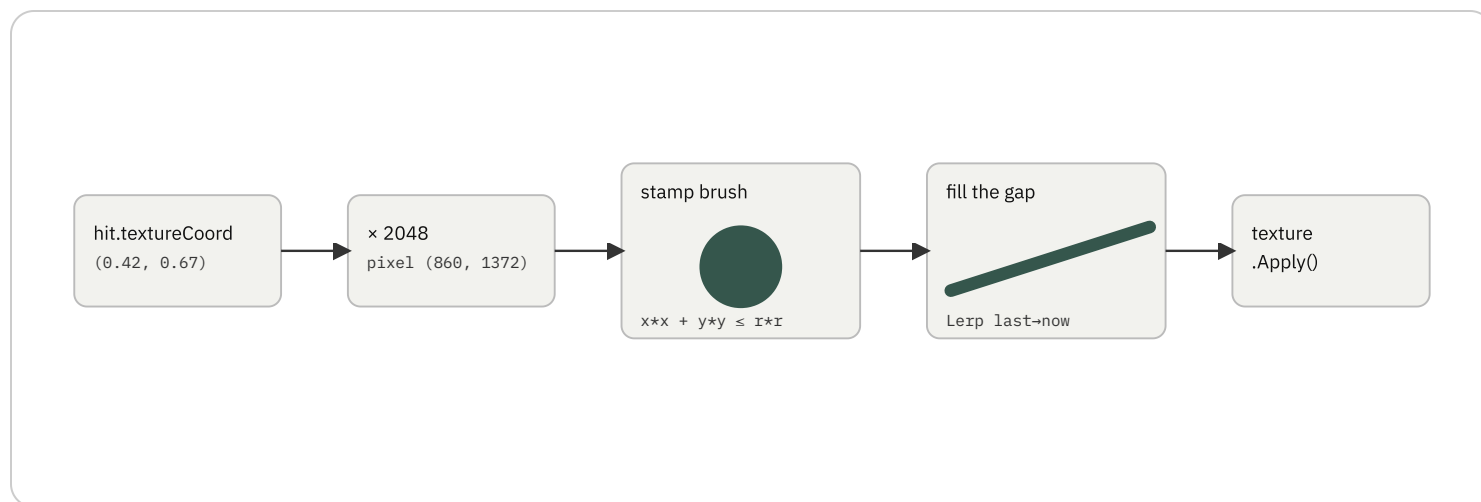


FIG. 7.1 From ray hit to visible ink. The "fill the gap" step is what makes strokes continuous — without it, fast movements leave a dotted line because the ray only reports one point per frame.

7.2 Set up the board

- 1 Select your **Whiteboard** quad. Give it a plain white **Material** (the script overwrites its texture at start).
- 2 **Add Component → Mesh Collider**. A quad already has one from its Mesh Filter; make sure `Convex` is off and it is not a trigger. The Mesh Collider is what provides `textureCoord` on a hit — a Box Collider does not.
- 3 **Add Component → Whiteboard Drawer** (script).

Position it flat against the wall. In this project the board sits at $y \approx 4.5$, rotated 180° on Y, scaled wide (4, 1.5, 1).

MUST-HAVES FOR DRAWING TO WORK

1) The collider is a **Mesh Collider** (gives UVs). 2) The material/mesh has proper UVs — Unity's built-in Quad and Plane do. 3) The texture's **Read/Write** is not an issue here because the script creates the texture itself.

ASSETS/WHITEBOARDDRAWER.CS — ATTACH TO THE WHITEBOARD QUAD

```
using UnityEngine;

public class WhiteboardDrawer : MonoBehaviour
{
    [Header("Drawing Settings")]
    public Color drawColor = Color.black;
    public int brushSize = 5; // radius in pixels

    private Texture2D drawTexture;
    private bool needsApply;

    private Vector2 lastUV; // where the last painted point was
    private bool hasLastUV; // are we mid-stroke?

    void Start()
    {
        // make a blank white 2048x2048 texture and show it on this surface
        drawTexture = new Texture2D(2048, 2048, TextureFormat.RGBA32, false);
        Color[] pixels = new Color[2048 * 2048];
        for (int i = 0; i < pixels.Length; i++) pixels[i] = Color.white;
        drawTexture.SetPixels(pixels);
        drawTexture.Apply();
        GetComponent<Renderer>().material.mainTexture = drawTexture;
    }

    void Update()
    {
        if (needsApply) { drawTexture.Apply(); needsApply = false; } // push changes to GPU
        // once/frame
    }

    // called by ControllerRayFollower every frame the trigger is held on the board
    public void DrawAtRaycastHit(RaycastHit hit)
    {
        Vector2 currentUV = hit.textureCoord;

        if (hasLastUV)
        {
            // paint a chain of dots from the last point to this one
            float dist = Vector2.Distance(lastUV, currentUV);
            int steps = Mathf.Max(1, Mathf.CeilToInt(dist * 2048 / brushSize));
            for (int i = 0; i <= steps; i++)
            {
```

```

        Vector2 p = Vector2.Lerp(lastUV, currentUV, (float)i / steps);
        PaintAtUV(p.x, p.y);
    }
}
else PaintAtUV(currentUV.x, currentUV.y);    // first dot of a new stroke

lastUV = currentUV;
hasLastUV = true;
}

// called when the trigger is released or the ray leaves the board
public void StopDrawing() => hasLastUV = false;

private void PaintAtUV(float u, float v)
{
    int cx = (int)(u * drawTexture.width);
    int cy = (int)(v * drawTexture.height);

    // stamp a filled circle of radius brushSize
    for (int x = -brushSize; x <= brushSize; x++)
    for (int y = -brushSize; y <= brushSize; y++)
    {
        if (x * x + y * y <= brushSize * brushSize)
        {
            int px = Mathf.Clamp(cx + x, 0, drawTexture.width - 1);
            int py = Mathf.Clamp(cy + y, 0, drawTexture.height - 1);
            drawTexture.SetPixel(px, py, drawColor);
        }
    }
    needsApply = true;
}

public void ClearBoard()
{
    Color[] pixels = new Color[drawTexture.width * drawTexture.height];
    for (int i = 0; i < pixels.Length; i++) pixels[i] = Color.white;
    drawTexture.SetPixels(pixels);
    needsApply = true;
    hasLastUV = false;
}
}

```

- **needsApply** batches all the pixel writes and does one `texture.Apply()` per frame in `Update`. `Apply()` uploads the image to the GPU and is expensive — calling it once per painted dot would tank the frame rate.
- **The eraser** is not a special mode — it just draws white with a big `brushSize` (see Part 8).
- **ClearBoard()** is there if you want to add a "wipe" button later.



FIG. 7.2 Drawing in the finished build. Hold the trigger and drag the laser across the board; the interpolation step keeps the stroke solid. (This build tinted the board green like a classroom chalkboard — the pen colour and board colour are both yours to choose.)

PART 8

The colour palette

Six small cubes under the board. Point and click one to change the pen colour; one of them is the eraser.

8.1 Build the palette

- 1 Create 6 small cubes (GameObject → 3D Object → Cube), scale `0.1`.
- 2 Name them **exactly**: `ColorPanel_Black`, `ColorPanel_Red`, `ColorPanel_Blue`, `ColorPanel_Green`, `ColorPanel_Yellow`, `ColorPanel_Eraser`. The script reads the colour from the name.
- 3 Add the **ColorPickerPanel** script to each. (Box Collider comes free with the cube.)
- 4 Create an empty **ColorPanelManager**, add **ColorPanelSetup**, and drag the 6 cubes into its `Panels` array. On Play it lines them up in a neat row.

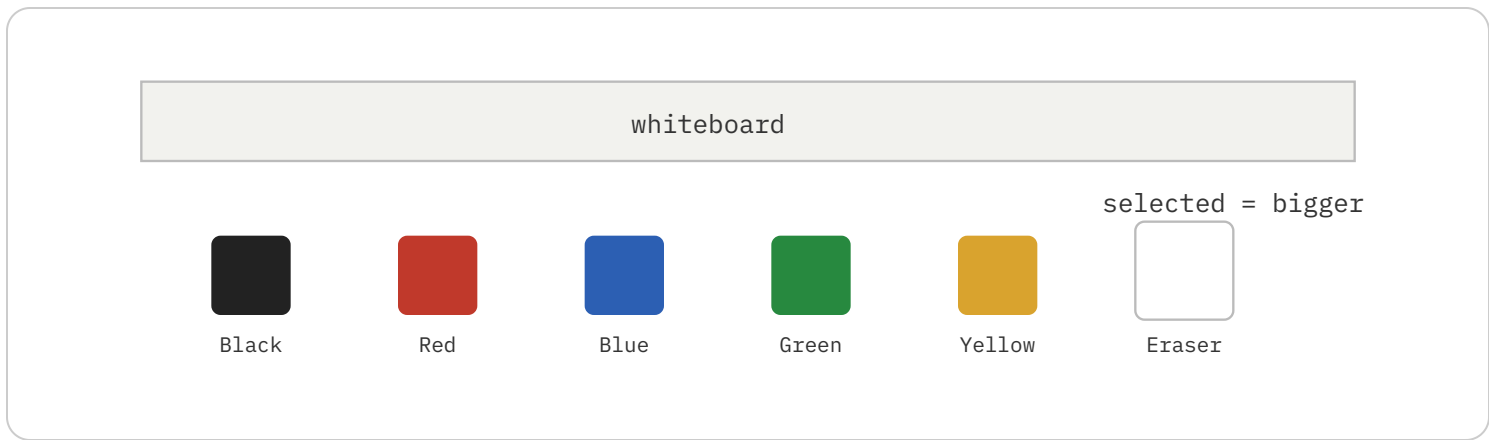


FIG. 8.1 Hovering a cube scales it to 1.1×; the selected cube sits at 1.2× so you can always see which colour is active. The eraser is white with a large brush.

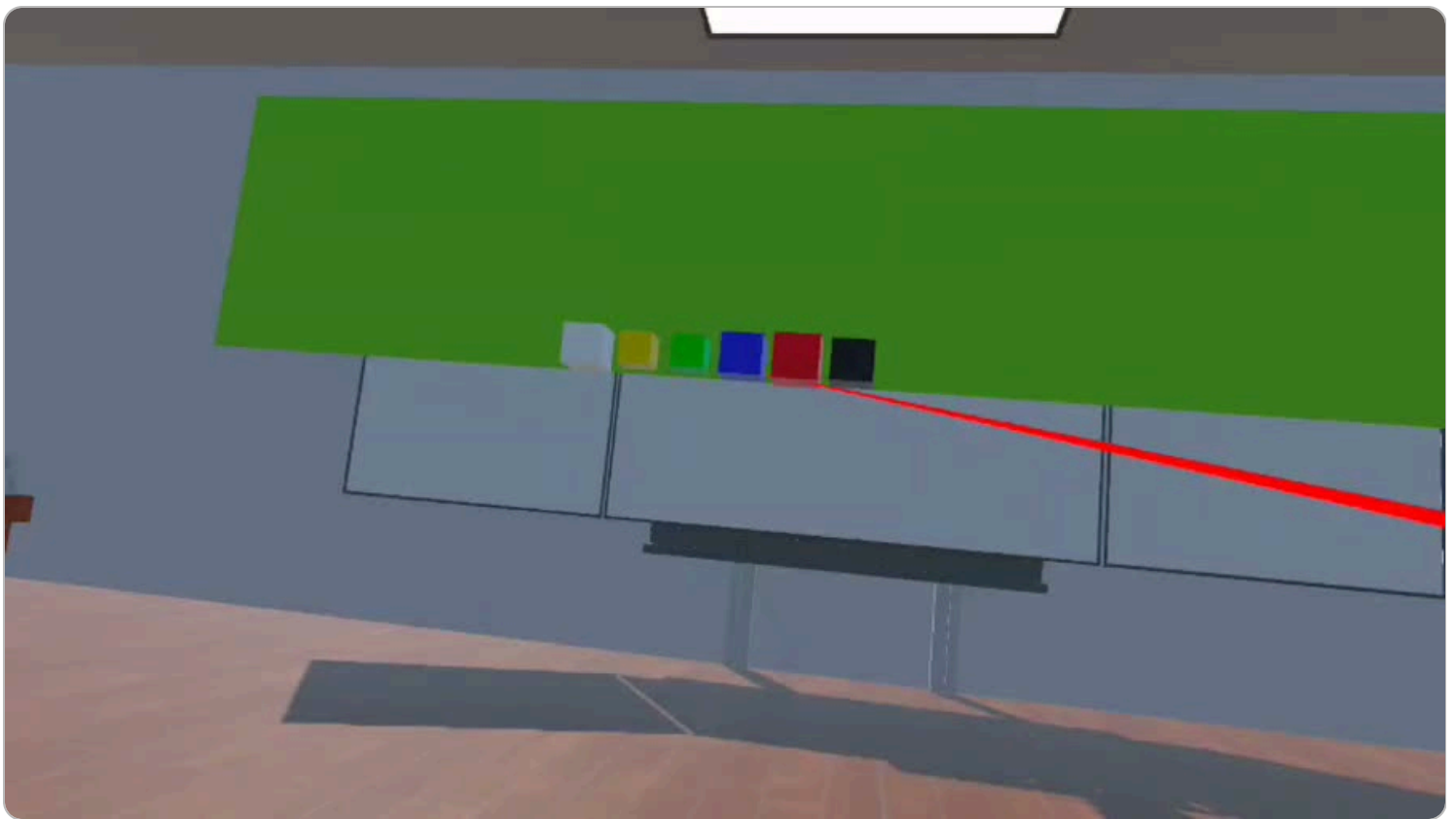


FIG. 8.2 The palette in the finished build — white (eraser), yellow, green, blue, red, black. Point the laser at one and pull the trigger to switch pen colour.

ASSETS/COLORPICKERPANEL.CS — ONE ON EVERY COLORPANEL_* CUBE

```
using UnityEngine;

public class ColorPickerPanel : MonoBehaviour
{
    private string colorName;
    private static ColorPickerPanel currentSelected; // shared across all panels

    void Start()
```

```

{
    // "ColorPanel_Red" -> "red"
    colorName = gameObject.name.Replace("ColorPanel_", "").ToLower();

    // tint the cube so the palette is colour-coded
    Color c = colorName switch
    {
        "black" => Color.black, "red" => Color.red,
        "blue" => Color.blue, "green" => Color.green,
        "yellow" => Color.yellow, "eraser" => Color.white,
        _ => Color.grey
    };
    GetComponent<Renderer>().material.color = c;
}

// called by ControllerRayFollower when you click this cube
public void SelectColor()
{
    WhiteboardDrawer drawer = FindObjectOfType<WhiteboardDrawer>();
    if (drawer == null) return;

    if (colorName == "eraser") { drawer.drawColor = Color.white; drawer.brushSize = 50; }
    else
    {
        drawer.drawColor = (colorName switch {
            "black" => Color.black, "red" => Color.red,
            "blue" => Color.blue, "green" => Color.green,
            "yellow" => Color.yellow, _ => Color.black });
        drawer.brushSize = 5;
    }

    // move the "selected" highlight from the old panel to this one
    if (currentSelected != null) currentSelected.SetSelected(false);
    currentSelected = this;
    SetSelected(true);
}

public void SetHover(bool on)
{
    if (currentSelected == this) return;
    transform.localScale = Vector3.one * (on ? 0.11f : 0.1f);
}

void SetSelected(bool on)
{
    transform.localScale = Vector3.one * (on ? 0.12f : 0.1f);
}
}

```

static currentSelected is shared by every panel, so there is only ever one selected cube. When you pick a new colour the previous cube shrinks back automatically.

```

using UnityEngine;

public class ColorPanelSetup : MonoBehaviour
{
    public GameObject[] panels;           // drag the 6 cubes here

    void Start()
    {
        float spacing = 0.15f, y = 1.18f, z = -11.5f;    // tune to your wall
        float startX = 2.8f - (spacing * 5f / 2f);

        for (int i = 0; i < panels.Length; i++)
        {
            if (panels[i] == null) continue;
            panels[i].transform.position = new Vector3(startX + i * spacing, y, z);
            panels[i].transform.localScale = Vector3.one * 0.1f;
            panels[i].transform.rotation = Quaternion.identity;
        }
    }
}

```

The y and z values are specific to where the wall is in this project's room model. Move one cube by hand to find good numbers, then copy them in.

PART 9

The cycling wall map

A framed map on the wall. Click it and it flips to the next image.

The room model has a **Map_Image** surface. The script swaps the texture on that surface each time you click. In the shipped project the images are five solid colours generated in code — placeholders you replace with real maps or slides.

- 1 Find the **Map_Image** object in the room hierarchy (it is a sub-mesh of the imported model).
- 2 Add the **MapImageCycler** script to it.
- 3 Make sure it has a **Collider** so the laser can hit it (add a Box Collider if needed).

```

using UnityEngine;

public class MapImageCycler : MonoBehaviour
{
    private Texture2D[] images;
}

```

```

private int      index;
private Renderer mapRenderer;

void Start()
{
    mapRenderer = GetComponent<Renderer>();

    // build 5 placeholder textures (swap these for real images / slides)
    Color[] colors = { Color.red, Color.blue, Color.green, Color.yellow, Color.cyan };
    images = new Texture2D[5];
    for (int i = 0; i < 5; i++)
    {
        var tex = new Texture2D(512, 512);
        var px  = new Color[512 * 512];
        for (int p = 0; p < px.Length; p++) px[p] = colors[i];
        tex.SetPixels(px); tex.Apply();
        images[i] = tex;
    }
    Apply(0);
}

void Apply(int i)
{
    // material slot 1 on this renderer is the map picture
    Material[] mats = mapRenderer.materials;
    mats[1].mainTexture = images[i];
    mapRenderer.materials = mats;
}

// called by ControllerRayFollower on click
public void OnLaserTrigger()
{
    index = (index + 1) % images.Length;
    Apply(index);
}
}

```

USING REAL IMAGES

Put your map images in `Assets`, tick **Read/Write Enabled** in each one's import settings, add a `public Texture2D[] realImages;` field, drag them in, and use that array instead of the generated colours. The `mats[1]` index depends on your model — check the material list on the renderer and pick the slot that is the map picture, not the frame.

The proximity warning system

When your head gets close to a desk or wall, the controllers buzz, a sound plays, and a warning appears in front of you.

10.1 Setup

- 1 Create the tag **Obstacle** (Inspector → Tag dropdown → Add Tag...).
- 2 Tag every desk, chair and wall you want to warn about as **Obstacle**, and make sure each has a Collider.
- 3 Make a **world-space UI Canvas** with a Text child ("Careful — step back"). Position it a metre or so in front of the camera. Call it `WarningText` / its canvas.
- 4 Create an empty **WarningSystem** GameObject, add the **ProximityWarningSystem** script, drag the warning canvas into its `Warning UI` slot, and (optionally) drop an alert `.mp3` into `Warning Sound`.

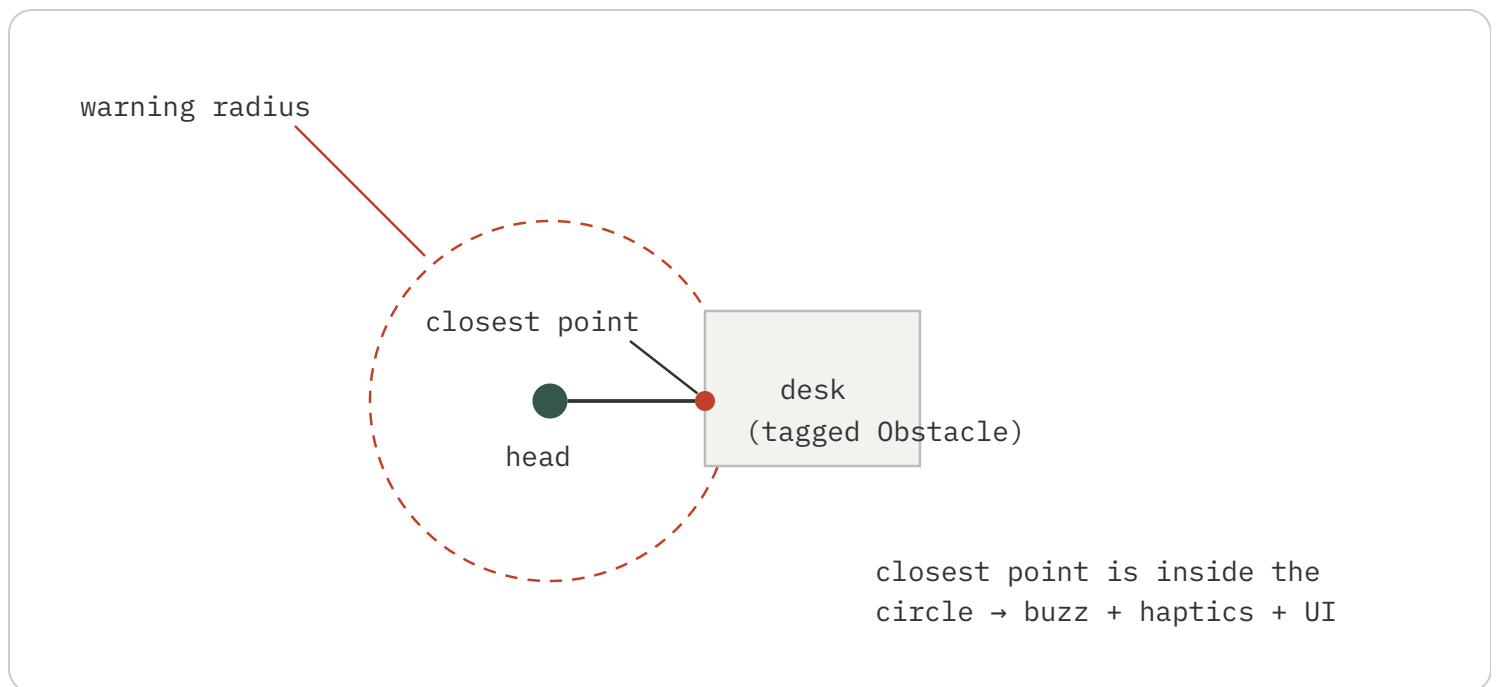


FIG. 10.1 Each frame the script asks every obstacle collider for its `ClosestPoint` to the head. If any is within `warningDistance` (and roughly the same height), the warning fires.

ASSETS/PROXIMITYWARNINGSYSTEM.CS — ON AN EMPTY WARNINGSYSTEM OBJECT

```
using UnityEngine;
using System.Collections.Generic;

public class ProximityWarningSystem : MonoBehaviour
```

```

{
    [Header("Warning Settings")]
    public float warningDistance = 2f;
    public float warningHeight = 2f;
    public GameObject warningUI;

    [Header("Feedback")]
    public float warningCooldown = 2f; // seconds between buzz/sound repeats
    public float hapticFrequency = 0.5f, hapticAmplitude = 0.8f, hapticDuration = 0.3f;
    public AudioClip warningSound;

    private Transform head;
    private List<Collider> obstacles = new List<Collider>();
    private AudioSource audioSource;
    private float lastWarningTime = -999f;
    private bool isWarningActive;

    void Start()
    {
        // the player's head is the rig's CenterEyeAnchor
        GameObject centerEye = GameObject.Find("CenterEyeAnchor");
        head = centerEye ? centerEye.transform : Camera.main.transform;

        audioSource = gameObject.AddComponent<AudioSource>();
        audioSource.playOnAwake = false;

        // collect every collider tagged "Obstacle"
        foreach (GameObject o in GameObject.FindGameObjectsWithTag("Obstacle"))
        {
            Collider c = o.GetComponent<Collider>();
            if (c != null) obstacles.Add(c);
        }

        if (warningUI != null) warningUI.SetActive(false);
    }

    void Update()
    {
        if (head == null || obstacles.Count == 0) return;

        bool tooClose = false;
        foreach (Collider ob in obstacles)
        {
            if (ob == null) continue;
            Vector3 p = ob.ClosestPoint(head.position);
            if (Vector3.Distance(head.position, p) < warningDistance &&
                Mathf.Abs(head.position.y - p.y) < warningHeight)
            { tooClose = true; break; }
        }

        if (tooClose && !isWarningActive) // just entered the zone
        {
            isWarningActive = true;
            if (warningUI != null) warningUI.SetActive(true);

            if (Time.time - lastWarningTime > warningCooldown) // rate-limit buzz/sound
            {
                lastWarningTime = Time.time;
                OVRInput.SetControllerVibration(hapticFrequency, hapticAmplitude,

```

```

OVRInput.Controller.LTouch);
        OVRInput.SetControllerVibration(hapticFrequency, hapticAmplitude,
OVRInput.Controller.RTouch);
        Invoke(nameof(StopHaptics), hapticDuration);
        if (warningSound != null) audioSource.PlayOneShot(warningSound);
    }
}
else if (!tooClose && isWarningActive)    // just left the zone
{
    isWarningActive = false;
    if (warningUI != null) warningUI.SetActive(false);
}
}

void StopHaptics()
{
    OVRInput.SetControllerVibration(0, 0, OVRInput.Controller.LTouch);
    OVRInput.SetControllerVibration(0, 0, OVRInput.Controller.RTouch);
}
}

```

- **ClosestPoint** gives the nearest spot on a collider's surface — more accurate than measuring to the object's centre.
- **The height check** stops the floor and ceiling from triggering it constantly.
- **The cooldown** separates the two feedbacks: the on-screen text shows the whole time you are too close, but the buzz and sound only repeat every 2 seconds so they are not annoying.
- **OVRInput.SetControllerVibration** is the Meta SDK's haptics call; `Invoke(nameof(StopHaptics), ...)` schedules the "stop" after `hapticDuration`.

PART 11

Test, build & deploy

Check it on your monitor first, then put it on the headset.

11.1 Testing without a headset

Two options while iterating:

- **Meta XR Simulator** — a fake headset window on your desktop. Enable it from **Meta → Meta XR Simulator → Activate**, then press Play. Move with WASD, look with the mouse.
- **A free-fly camera** for checking the room layout. This project had a small helper that lets you fly around with WASD + mouse in Play mode — handy for placing props before you have the headset on:

```

using UnityEngine;

public class FreeCameraController : MonoBehaviour
{
    public float moveSpeed = 5f, lookSpeed = 2f;
    private float rotX, rotY;

    void Start() => Cursor.lockState = CursorLockMode.Locked;

    void Update()
    {
        // mouse look
        rotX = Mathf.Clamp(rotX - Input.GetAxis("Mouse Y") * lookSpeed, -80f, 80f);
        rotY += Input.GetAxis("Mouse X") * lookSpeed;
        transform.rotation = Quaternion.Euler(rotX, rotY, 0);

        // WASD + E/Q for up/down
        Vector3 move = transform.forward * Input.GetAxis("Vertical")
            + transform.right * Input.GetAxis("Horizontal")
            + Vector3.up * ((Input.GetKey(KeyCode.E) ? 1 : 0) - (Input.GetKey(KeyCode.Q) ? 1
: 0));
        transform.position += move * moveSpeed * Time.deltaTime;

        if (Input.GetKeyDown(KeyCode.Escape)) Cursor.lockState = CursorLockMode.None;
    }
}

```

Put this on a spare Camera, not on the VR rig. Disable or delete it before the real build so it does not fight the headset tracking.

11.2 Build to the Quest

This assumes the computer already sees the headset. If `adb devices` does not list it, go back to §1.3 — the setup differs by operating system.

- 1 Plug the headset in with USB-C. Put it on and accept **Allow USB debugging** if asked.
- 2 **File → Build Settings**. Confirm platform is **Android** and `SampleScene` is ticked in the list.
- 3 In **Run Device**, pick your headset by name. (If it is missing: **Refresh**, re-check developer mode, try another cable/port.)
- 4 Click **Build And Run**. Choose a filename for the `.apk`. First build takes 5–15 minutes.
- 5 When it finishes it installs and launches automatically. Later you will find it in the headset under **Library → Unknown Sources**.

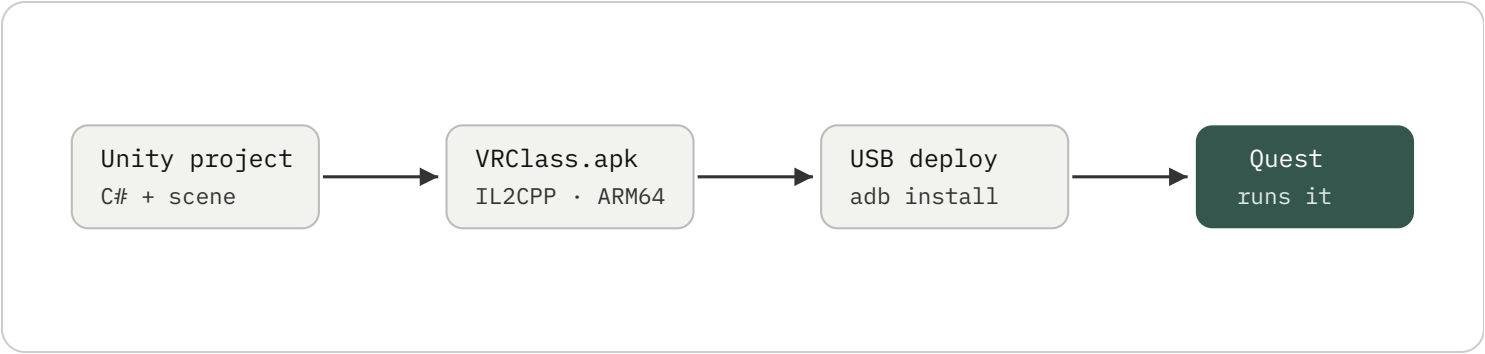


FIG. 11.1 "Build And Run" does all four steps. "Build" alone just makes the .apk, which you can sideload later with SideQuest.

DEMO CHECKLIST FOR YOUR PROFESSOR

Walk in → point at the board → pick red → write a word → switch to eraser → wipe it → grab a chair from across the room and drop it → click the wall map twice → walk toward a desk until it buzzes. That is every feature in about 40 seconds.

APPENDIX

Reference

A Script map — what goes where

SCRIPT	ATTACH TO	NEEDS ALSO	PUBLIC METHODS CALLED BY THE LASER
ControllerRayFollower	LaserPointer (child of RightHandAnchor)	Line Renderer	— (it is the caller)
VRGrabbableObject	Every grabbable prop	Collider, Rigidbody	Grab() , Release() , OnHoverEnter/Exit()
WhiteboardDrawer	Whiteboard quad	Mesh Collider, a material	DrawAtRaycastHit() , StopDrawing()
ColorPickerPanel	Each ColorPanel_* cube	Collider	SelectColor() , SetHover()
ColorPanelSetup	ColorPanelManager (empty)	—	— (runs once on Start)
MapImageCycler	Map_Image surface	Collider	OnLaserTrigger()
ProximityWarningSystem	WarningSystem (empty)	Obstacle tag + world-space UI	— (polls every frame)

SCRIPT	ATTACH TO	NEEDS ALSO	PUBLIC METHODS CALLED BY THE LASER
<code>FixMaterials</code> (<i>editor</i>)	lives in <code>Assets/Editor/</code>	—	menu: Tools > Fix Everything

B Troubleshooting

SYMPTOM	LIKELY CAUSE & FIX
Everything is grey / white after import	Textures not connected. Run Tools → Fix Everything , or drag each texture into its material's Base Map by hand.
Laser goes through objects, nothing reacts	The object has no Collider, or its Collider is <code>Is Trigger</code> . Add a solid collider.
Drawing shows dots, not lines	Fast strokes with no interpolation — make sure you kept the <code>Lerp</code> loop in <code>DrawAtRaycastHit</code> .
Board does nothing when I point at it	It has a Box Collider, not a Mesh Collider. Only Mesh Colliders return <code>textureCoord</code> .
Grabbed objects jitter or fly away	Grab logic is in <code>Update</code> instead of <code>FixedUpdate</code> , or <code>followSpeed</code> is too high.
Headset not listed in Build & Run	Developer mode off, USB-debugging prompt not accepted, or a charge-only cable. Try <code>adb devices</code> in a terminal.
Black screen on the Quest after launch	XR Plug-in Management → Android tab → Oculus not ticked; or Scripting Backend not IL2CPP / ARM64.
Compiles but <code>OVRInput</code> / <code>OVRCameraRig</code> unknown	Meta XR SDK not imported, or its assembly not referenced. Re-import from Package Manager.
Warning never triggers	Obstacles not tagged <code>Obstacle</code> , or they have no Collider, or the script's <code>CenterEyeAnchor</code> lookup failed (check the name matches your rig).

C Assets used

Every 3D model in the project is a free download. Replace any of them with your own — nothing in the scripts depends on a specific model, only on the components you attach.

IN THE ROOM	FILE	SOURCE TYPE
The classroom shell	<code>Brand new Study Room.fbx</code>	Free Sketchfab model (with texture set)
Bookcase	<code>BookCase1.fbx</code> , aquarium bookcase	Free download
Books & inkwell	<code>BookA.fbx</code> , <code>Inkwell.fbx</code> , antique book	Free download
Waste bin	<code>SM_Big_bin_embedded.fbx</code>	Free download

IN THE ROOM	FILE	SOURCE TYPE
School bouquet	SchoolBouquetFixed.fbx	Free download
Quest 2 model (on a desk)	oculus_quest_2.fbx	Free download
Car (outside the window / prop)	AUDI_Q3_SPORTBACK.fbx	Free download
Character model	Spidey.fbx	Free download

Keep a text file in your project listing each model's page URL and licence — most free models ask for credit. Your professor will likely want to see it.

D Glossary of the VR-specific bits

OVRCameraRig	The Meta component that turns a GameObject tree into a headset + hands.
OVRInput	Reads the Quest controllers. <code>GetDown</code> / <code>GetUp</code> / <code>Get</code> for "pressed"/"released"/"held", plus <code>SetControllerVibration</code> for haptics.
rightHandAnchor	A Transform inside the rig that Unity keeps on your real right controller.
Building Block	A ready-made prefab from Meta → Tools → Building Blocks — the Camera Rig is one.
Raycast	An invisible line from A in direction B; <code>Physics.RaycastAll</code> returns everything it crosses.
UV / textureCoord	A 2D coordinate (0–1, 0–1) saying where on a texture a point on a 3D surface is.
Haptics	Controller vibration.
Sideload	Installing an app on the Quest outside the official store (what "Build And Run" does).

VRClass — A Beginner's Field Guide. Created by **Sara Mjeshtri** and **Jorida Lilollari**. Written from the finished project: Unity 2022.3.62f3, Built-in pipeline, Meta XR SDK 201.0.0, target Meta Quest 2/3/Pro. One scene, six gameplay scripts, one editor tool.

To save this as a PDF: use your browser's **Print → Save as PDF** (choose "Default" margins, background graphics on).